AutoML

Machine Learning, Simplified.

An End-to-End Automated Machine Learning Platform for Data Analysis & Model Training

Project

AutoML Web Platform

Team

- P Charan
  240150024
- M Charan
  240150018
- Lalit Deshmane
  240150044

Course

DA 213 - Python Programming Lab

Problem Statement & Objectives

AutoML

The Problem

- Data scientists face significant challenges across the entire ML pipeline
- Manual model selection, hyperparameter tuning, and preprocessing are time-consuming
- Non-technical users lack tools to leverage machine learning effectively
- Significant gap remains between raw data and actionable ML models

Our Objectives

- Automate end-to-end ML workflow from data upload to model deployment
- Simplify complex ML processes with an intuitive user interface
- Enable real-time inference and predictions on new data
- Provide deep exploratory data analysis (EDA) capabilities automatically
- Facilitate feedback collection and continuous improvement

System Architecture & Workflow

The platform follows a clean separation of concerns: a React.js frontend sends requests through a REST API to a FastAPI backend, which coordinates the data pipeline, ML engine, and inference modules before saving results to storage.

Frontend

React.js with Glassmorphism CSS for a modern, responsive UI

Backend

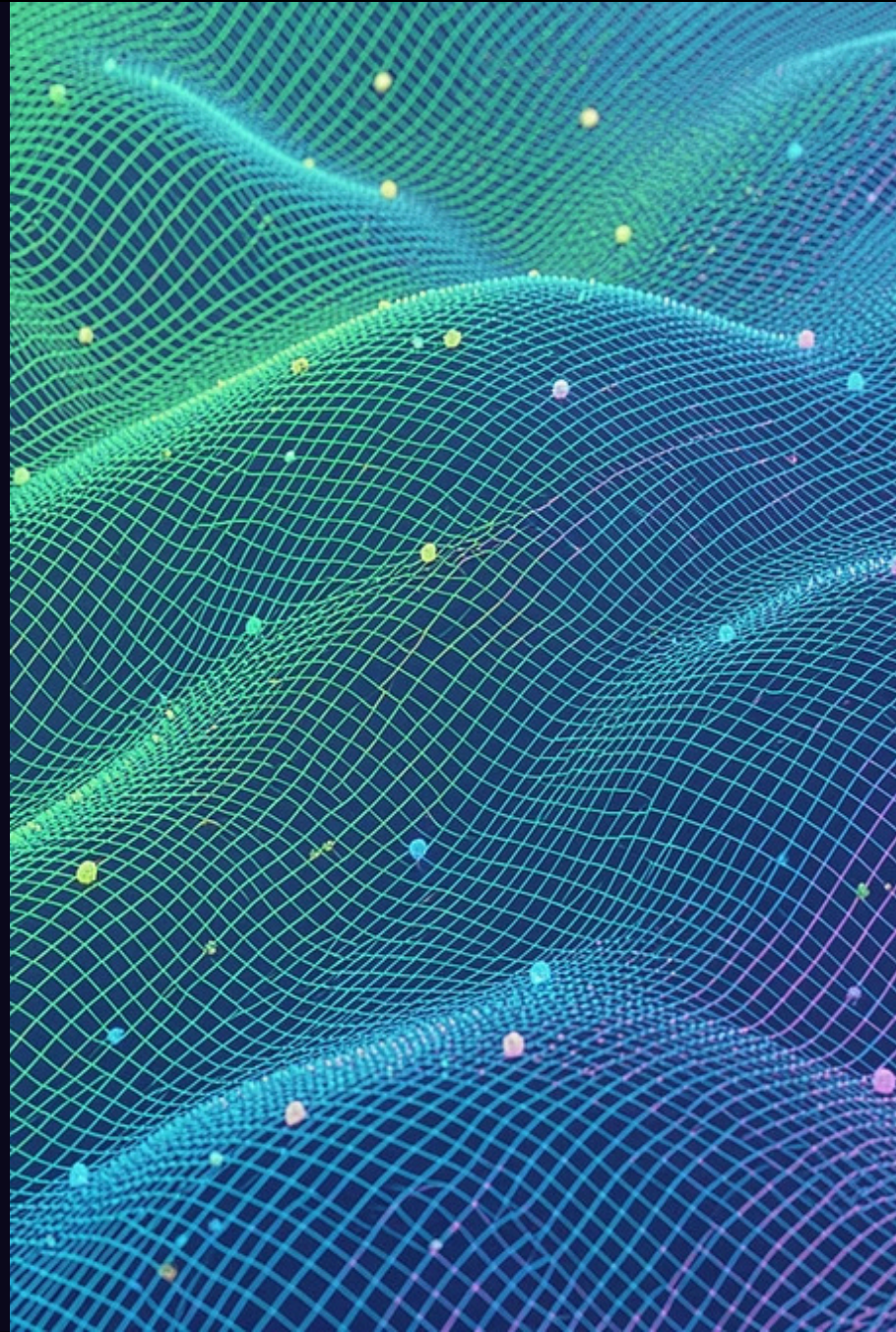
FastAPI Python async framework for high-performance request handling

ML Libraries

Scikit-learn, Pandas, NumPy powering all model training and preprocessing

Communication

REST API (JSON) with Resend API for email-based feedback delivery



Machine Learning Approach

The platform follows a clean AutoML pipeline: data is ingested and validated, analyzed for readiness, preprocessed for modeling, and then routed into a model selection strategy that compares algorithms with cross-validation.

Data Ingestion & Validation

Accept CSV files with flexible schema, validate data integrity and format, generate initial data statistics

Exploratory Data Analysis

Automatic outlier detection (IQR), correlation analysis, distribution analysis, missing value identification, and ML Readiness Score calculation

Data Preprocessing

Column type inference, missing value imputation (mean/mode), LabelEncoder for categoricals, redundant feature removal, standard scaling

Model Selection Strategy

Auto-detect regression vs classification, evaluate multiple algorithms in a single pass, 5-fold cross-validation for robust ranking

ML Models Used

AutoML

CLASSIFICATION

Logistic Regression
Baseline linear classifier

Random Forest Classifier
Ensemble of decision trees

Gradient Boosting Classifier
Sequential boosting ensemble

AdaBoost / Extra Trees / HistGBM / KNN
Additional ensemble and instance-based models

Selection Criteria: Accuracy & F1-Score (weighted), cross-validation ranking

REGRESSION

Linear Regression
Baseline linear regressor

Random Forest Regressor
Ensemble of decision trees

Gradient Boosting Regressor
Sequential boosting ensemble

AdaBoost / Extra Trees / HistGBM / KNN
Additional ensemble and instance-based models

Selection Criteria: R^2 Score, MAE, cross-validation ranking

Feature importance is extracted from tree-based models, enabling top feature visualization and correlation analysis with the target variable — giving users interpretable insights alongside predictions.

Dataset Info And Preprocessing

AutoML

Supported Data Format

- CSV (Comma-Separated Values) files
- Flexible schema (automatically detects columns)
- Handles both numeric and categorical data

Dataset Features

- Column type auto-detection (numeric vs categorical)
- Mixed data type support
- Missing value handling
- Outlier detection and reporting

Data Quality Metrics

- ML Readiness Score (combines multiple factors)
- Missing value percentage per column
- Outlier count and percentage
- Feature uniqueness and cardinality

Example Datasets Used

- 1000 Companies Dataset (regression/classification)
- AI Job Impact Dataset
- Titanic Dataset (classification benchmark)

Preprocessing Techniques

For Numeric Features

- Missing value imputation (mean strategy)
- Outlier detection using IQR method
- Standard scaling normalization
- Range: [-∞, +∞] → [0, 1]

For Categorical Features

- Missing value imputation (mode strategy)
- Label encoding for ML compatibility
- Unique value cardinality analysis
- Redundant feature removal (>15 unique values in string cols)

Technology Stack & Tools

AutoML

Frontend

- **React.js 18** — UI component library
- **CSS3 + Glassmorphism** — modern responsive design
- **Fetch API** — HTTP communication
- **React Hooks** — component state management
- **Custom JavaScript** — data visualization

Backend

- **FastAPI** — async HTTP server
- **Python 3.10+** — modern language features
- **Pandas / NumPy** — data processing
- **Scikit-learn** — ML algorithms
- **ThreadPoolExecutor** — parallel training
- **Pickle** — model serialization
- **Resend API** — feedback email delivery

Infrastructure

- **Docker** — containerization for consistent deployment
- **Hugging Face Spaces** — production deployment
- **REST API** — frontend-backend communication
- **CORS** — cross-origin security
- **Git** — version control
- **venv / npm** — dependency management

Core System Modules

AutoML

Backend Modules

1

model.py — ML Engine

train_model(), predict_model(), predict_dataset(), _detect_problem_type(), _infer_column_types(), _find_redundant_features(), _prepare_predict_input()

2

main.py — API Server

/upload · /train · /predict · /predict-bulk · /eda · /feedback · /history · /status

3

Data Pipeline

CSV parsing, missing value handling, outlier detection (IQR), feature scaling & encoding, 80-20 train-test split

Frontend Modules

1

Pages

Home.js · Upload.js · Results.js · Predictor.js · History.js

2

EDA Dashboard

SummarySection · DistributionSection · CorrelationSection · OutlierPanel · FeatureImportance · MissingValues · ModelScores

3

Components & Utilities

Layout.js · FeedbackForm.js · api.js · helpers.js · constants.js

Results and Performance Outputs

AutoML

Classification Metrics

- Accuracy Score (0–100%)
- F1-Score (weighted average)
- Cross-validation scores
- Feature importance rankings
- Confusion matrix on dashboard

Regression Metrics

- R^2 Score (coefficient of determination)
- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Cross-validation scores
- Residual analysis

EDA Visualizations

- Distribution histograms for all numeric columns
- Box plots for outlier visualization
- Correlation heatmap (numeric features)
- Feature importance bar charts
- Missing value % by column
- Categorical value distributions

Export Options

- Model download (pickle format)
- Batch predictions export (CSV)
- EDA report (JSON with charts)
- Training metadata (features, scalers, encoders)

Prediction Results: Single prediction with confidence scores, batch predictions downloadable as CSV, feature contributions to prediction, and model uncertainty estimates where available.

User Workflow

AutoML

How Users Interact with the System

1. Upload Data

- User uploads a CSV file
- System validates and shows a quick preview

2. Train Model

- User selects target column
- Clicks "Train Model"
- Model is trained automatically

3. Explore Insights (EDA)

- View interactive dashboard
- Data summary & readiness score
- Feature distributions & correlations
- Feature importance

4. Make Predictions

- Single Prediction: Fill form → get instant result
- Bulk Prediction: Upload CSV → download predictions

5. Provide Feedback

- Submit feedback via form
- Receive confirmation

Demo Examples: Classification: Titanic dataset → predict survival. Regression: House prices → estimate price.

AutoML

Current Limitations:

Data Limitations

- CSV format only (no JSON, Parquet, Excel currently)
- Maximum file size depends on server memory

Model Limitations

- Fixed set of 7 algorithms per task type
- No custom hyperparameter tuning UI
- Limited to supervised learning (no clustering, dimensionality reduction)
- Single model per training session
- No model versioning or comparison

Feature Limitations

- No data augmentation options
- No feature engineering recommendations
- Limited to default preprocessing strategies

Future Scope & Enhancements:

Phase 1 (Short-term)

- Support for multiple file formats (Excel, JSON, Parquet)
- Hyperparameter tuning interface for advanced users
- Model comparison dashboard (A/B testing)
- More detailed model explainability (SHAP values)
- Custom preprocessing pipeline builder
- Data profiling and quality scoring

Phase 2 (Medium-term)

- Neural network support (TensorFlow/PyTorch)
- Time-series forecasting models
- Unsupervised learning (clustering, dimensionality reduction)
- Automated feature engineering recommendations
- Class imbalance handling (SMOTE, class weights)
- Model deployment as API endpoints
- Model versioning and rollback

Phase 3 (Long-term)

- Multi-modal data support (images, text, audio)
- Advanced NLP capabilities
- Automated ML (AutoML) hyperparameter optimization
- Graph neural networks for relational data
- Reinforcement learning capabilities
- Real-time streaming data support
- Custom model marketplace integration

References

AutoML

Core Technologies

- Scikit-learn, Pandas, NumPy
- FastAPI, React.js

Key Resources

- Official Documentation (Scikit-learn, FastAPI, React, Docker)
- Hands-On Machine Learning – Aurélien Géron
- Introduction to Statistical Learning

Datasets

- Kaggle
- UCI ML Repository

GitHub

- Related AutoML Tools
- Auto-Sklearn
- TPOT
- H2O AutoML
- Google AutoML

Project & Tools

- Python, Node.js, Git
- Hugging Face Spaces, Resend API
- GitHub Repository